DAILY JAVA DEEP DIVES · THE LATEST NEWS, FULLY EXPLAINED

Java – Beautifully Broken

Structured concurrency finally graduates, and Jakarta EE learns to wake up from a snapshot

Issue #14 | Mon, 19 Oct 2026 | ⏱️ ~42 min read | Level 3/5

☕ **Today's two stories**

1. **Structured concurrency is now a permanent part of JDK 28.** On Saturday, 17 October, the code that makes `StructuredTaskScope` a final API landed in the JDK's main line, ending two incubator rounds and seven previews. We diffed the commit: the public API is exactly the JDK 27 one, but a few edge-case behaviours changed. Here's what "final" really means, what changed under the hood, and how to move your code from the JDK 21 and JDK 25 previews.
2. **Jakarta EE is getting a standard for checkpoint and restore.** On 13 October the Jakarta EE Specification Committee approved a brand-new specification, Jakarta CRaC, which defines how application servers take part in JVM snapshots so your app can resume in milliseconds instead of booting from scratch. We explain CRaC from the ground up, how the new spec plans to use it, how it compares with Project Leyden, and where Jakarta EE 12 stands six weeks before its first release.
3. **On the radar:** 5 more things that happened, listed at the end.

STORY 1 Structured concurrency graduates in JDK 28

JDK 28 · OPENJDK · PROJECT LOOM · JEP 543 INTEGRATED 17 OCT 2026 · ~15 MIN READ

Ten days ago, in issue #5, we described structured concurrency as a feature that was "about to" become a normal part of Java: a Candidate JEP with an open pull request and a review still to come. That "about to" is over. Early on Saturday morning, a 12-file commit removed the word "preview" from `java.util.concurrent.StructuredTaskScope` for good. After four years, two incubators and seven previews, the API that lets you treat a bundle of threads as a single task is finished. Here's what happened, what quietly changed in the last mile, and what it means for code you write today.

WHAT HAPPENED

On 17 October 2026 at 06:06 UTC, Oracle's Alan Bateman integrated **"8391460: Implement JEP 543: Structured Concurrency"** into the JDK main line, reviewed by Viktor Klang.[1] It changes 12 files (612 lines added, 434 removed). The headline changes are short: the `@PreviewFeature` annotation is removed from `StructuredTaskScope` and from `StructureViolationException`, their `@since` tags become `@since 28`, and the JDK's internal list of preview features no longer marks structured concurrency as "Seventh Preview".[1]

This is the last step of a process we followed in issue #5. **JEP 543** *≈ the JDK Enhancement Proposal to make structured concurrency a final, permanent feature* proposed to finalise the API "without change, after seven rounds of preview". InfoQ reported that it moved from Candidate to **Proposed to Target** *≈ the stage where a JEP is formally*

proposed for a specific JDK release, followed by a short public review period for JDK 28, with the review "expected to conclude on Tuesday, October 13, 2026".[2] Four days after the review window closed, the code was in. (As of 19 October, Inside.java hadn't yet posted a "targeted to JDK 28" announcement for JEP 543, as it did for JEP 545 on 16 October.[3])

To be precise about where this lives: it's in the **main line** *≈ the master branch of the openjdk/jdk repository, from which the next JDK release is built*, not yet in a downloadable build. The newest JDK 28 **early-access build** *≈ a weekly pre-release JDK binary published so developers can test upcoming features* we found, build 20, was tagged on 15 October and predates the commit, so you'll need the next build to try it.[4] JDK 28 itself is a non-**LTS** *≈ Long-Term Support: a JDK release that vendors support for years (21, 25, the planned 29); the releases in between get about six months* release due in March 2027.[5]

THE BACKGROUND YOU NEED

What structured concurrency is, in one paragraph

If you read issue #5, skip ahead. If not, here's the idea. A **thread** *≈ an independent path of execution inside your program* that starts other threads normally has no relationship with them: if one fails, the others keep running, and nothing guarantees they've finished when your method returns. **Structured concurrency** *≈ a style of concurrent programming in which tasks started inside a block of code must finish before that block ends, just as a method call finishes before its caller continues* fixes that. You open a **scope** *≈ a block of code that owns a group of concurrent tasks and doesn't end until all of them have ended*, `fork()` some **subtasks** *≈ the pieces of work started inside a scope, each running in its own thread*, wait for them at one point with `join()`, and when the block ends, every thread it started has ended too. If a subtask fails, the others are cancelled. The thread that opened the scope is its **owner** *≈ the thread that opened a scope; only it may fork, join and close it*.

It only became practical once Java got **virtual threads** *≈ lightweight threads managed by the JVM rather than the operating system, cheap enough to create one per task* in JDK 21, because a scope typically starts a brand-new thread for every subtask.

Why "final" matters so much

New Java APIs usually arrive as a **preview API** *≈ a complete but not yet permanent feature that only works if you pass --enable-preview to javac and java, so it can still change* first. Previews are deliberately awkward to use in production, and the awkwardness is enforced in the class file itself. We tried it on JDK 21: we compiled a tiny program that uses the JDK 21 structured concurrency preview with `javac --release 21 --enable-preview`, and `javap` shows the class file version as **65.65535**.[6] That 65535 **minor version** *≈ the second half of a class file's version number; 65535 (all 16 bits set) marks a class that uses preview features* is the marker. Running the class without the flag fails immediately:

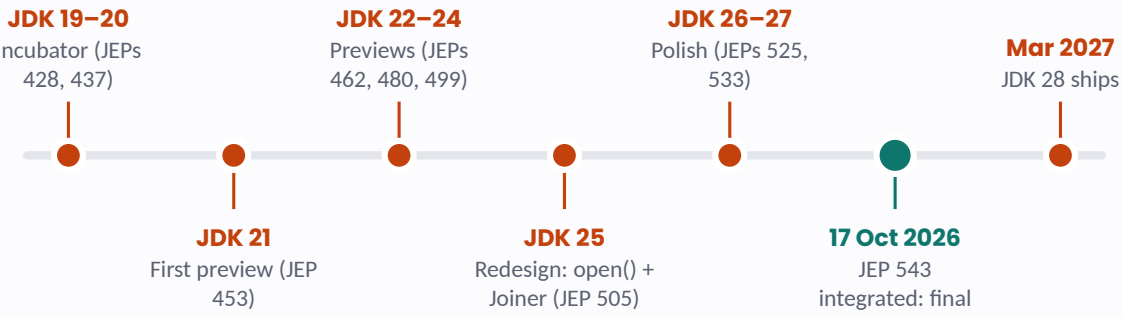
```
java.lang.UnsupportedClassVersionError: Preview features are not enabled
for Dash (class file version 65.65535). Try running with '--enable-preview'
```

The rules are spelled out in the original JVM work for preview features: a preview class file loads only if `--enable-preview` is given **and** its major version is the current release. Preview class files from an earlier release fail even with the flag.[7] So a library compiled against the JDK 27 preview simply won't load on JDK 28. That's why frameworks and library authors have largely stayed away from structured concurrency: depending on it meant forcing every user onto one exact JDK version, with a scary flag.

Now that it's final, code that uses `StructuredTaskScope` compiles to an ordinary class file, runs without flags and keeps running on every later JDK. That's the real news.

Four years, nine rounds

From incubator to final



Sources: JEP list in mammb's JDK 28 overview; InfoQ (12 Oct 2026); openjdk/jdk commit cd37d2b; InfoWorld (JDK 28 due March 2027).

The JEP numbers above come from a JDK 28 overview by Japanese blogger mammb, which lists the path from JEP 428 (JDK 19, incubator) through JEP 533 (JDK 27, seventh preview) to JEP 543.[8] The design changed a lot on the way. In the JDK 21 preview you created a scope with a constructor and picked a policy by choosing a subclass such as `ShutdownOnFailure`. JDK 25 (JEP 505) replaced that with static `open()` methods and a separate policy object, the `Joiner`. JDK 26 and 27 then polished exceptions and types: JEP 533, for example, made the standard joiners throw the familiar checked `ExecutionException` instead of a preview-only `FailedException`, and added a third type parameter for the exception `join()` throws.[9]

HOW IT WORKS

The API you get is the JDK 27 API, exactly

We compared the public declarations in `StructuredTaskScope.java` before and after the commit: all 27 of them (the interface, its nested `Subtask`, `Joiner` and `Configuration` types, every factory method, `fork`, `join` and so on) are identical.[1] So JEP 543 kept its promise of "no change". If your code compiles against the JDK 27 preview, it will compile against JDK 28 without the flag.

Here's the shape of the final API, written the way the JDK's own Javadoc examples are:[1]

```
// JDK 28: final API, no --enable-preview needed.
Response handle() throws InterruptedException, ExecutionException {
    try (var scope = StructuredTaskScope.open()) {
        Subtask<String> user  = scope.fork(() -> findUser());
        Subtask<Integer> order = scope.fork(() -> fetchOrder());
        scope.join();   // waits; throws if either subtask failed
        return new Response(user.get(), order.get());
    }
}
```

`open()` uses the default policy: wait for all subtasks to succeed, and if any fails, cancel the others and make `join()` throw an `ExecutionException` whose cause is the failure. Other policies come from **Joiner** *≈ an object you pass to open() that decides when a scope is done and what join() returns or throws* factory methods: `allSuccessfulOrThrow()` returns a `List` of results, `anySuccessfulOrThrow()` returns the first success and cancels the rest, and `allUntil(predicate)` lets you decide when enough is enough.[1] A second `open()` overload takes a **UnaryOperator** *≈ a function that takes a value and returns a value of the same type; here, it takes the default configuration and returns your adjusted one* to configure a timeout, a name or a custom **ThreadFactory** *≈ an object that creates the Thread objects a scope runs subtasks in; the default creates virtual threads*.

What actually changed in the last commit

"No API change" doesn't mean nothing changed. Most of the 671 lines touched in `StructuredTaskScope.java` are Javadoc rewrites: the usage rules are now a bulleted list (only the owner may fork, join and close; `join()` must be called after forking; neither `fork` method may be called after `join()`), and the cancellation section now says explicitly that cancelling interrupts the threads running unfinished subtasks.[1] But the commit also fixes three edge cases in the implementation class, `StructuredTaskScopeImpl`, and adds tests for each. If you write your own `Joiner`, the first one matters.

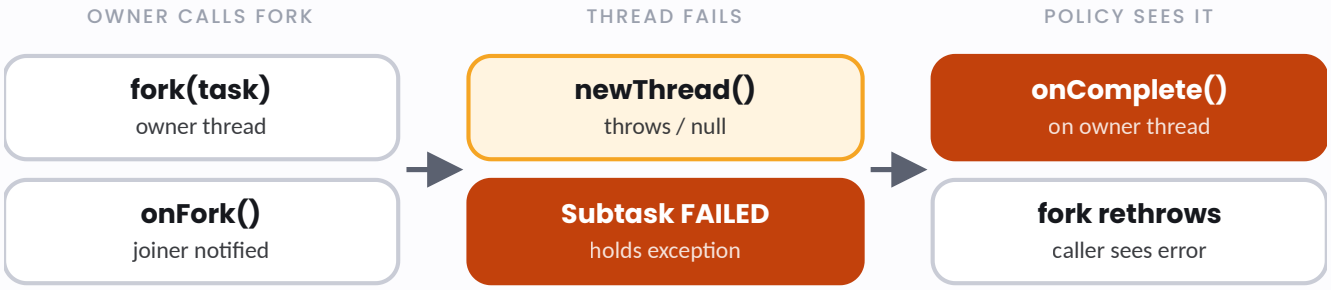
1. A fork that can't start a thread now counts as a failed subtask. Before, if the scope's `ThreadFactory` threw an exception or returned `null`, `fork()` threw and the subtask was left in limbo with no result. And if the underlying thread group refused to start the thread (for reasons other than cancellation), that was silently ignored. Now, in all of those cases, the subtask is marked `FAILED` with the exception, the joiner's `onComplete()` callback is called **on the owner thread** so the policy sees the failure, and only then does `fork()` rethrow. If `onComplete()` itself throws, that exception is attached to the original as a **suppressed exception** *≈ an extra exception attached to the main one with addSuppressed(), so it isn't lost*.[1] New tests named `testOnCompleteAfterForkFails` and `testOnCompleteThrowsAfterForkFails` pin this down.

2. A joiner can read the failure from inside that callback. Normally `Subtask.exception()` throws `IllegalStateException` unless you're the subtask's own thread or the owner has already joined. Because of change 1, the owner can now be inside `onComplete()` before any join, so the implementation records which thread is making the callback and lets exactly that thread call `exception()`.[1]

3. "Closed" now remembers whether you joined. The scope's internal state used to have a single `CLOSED` value, and its "has join completed?" check was `state >= JOIN_COMPLETED`, which was also true for a scope

that had been **closed without ever joining**. The commit splits it into `CLOSED_BEFORE_JOIN` and `CLOSED_AFTER_JOIN`.[1] The new tests check that after closing a scope that never joined, a subtask's result is still unavailable, even after repeated `close()` calls. In other words, you can't sneak a result out of a scope you abandoned.

fork() when the thread can't be created (JDK 28 final)



From StructuredTaskScopeImpl.fork() and SubtaskImpl.notRun() in openjdk/jdk commit cd37d2b. Before this commit, the subtask was left without a result and onComplete() was never called.

Under the hood: a "flock" of threads

How does a scope know which threads are its own, and how does it make the guarantee that nothing outlives it? Reading the implementation answers that, and it's worth knowing because the Loom team has hinted that this machinery may become public one day.

Every scope is backed by a **ThreadFlock** *≈ an internal JDK class (jdk.internal.misc.ThreadFlock) that groups threads running closely related tasks and waits for all of them*. Its Javadoc describes it as "a grouping of threads that typically run closely related tasks", where threads started in a flock stay in it until they terminate. It has the operations you'd expect: `open`, `start(thread)`, `awaitAll()`, `wakeup()` (to make `awaitAll()` return early, used for cancellation), `shutdown()` (no new threads) and `close()`, which "blocks until all threads in the flock finish".[10]

Here's how the scope's operations map onto it, from the source:[1,10]

- `open()` creates a flock owned by the current thread. The flock captures the owner's **scoped value** *≈ an immutable value bound for a bounded stretch of code and visible to everything it calls, including child threads in a scope (final since JDK 25)* bindings at that moment, so every subtask inherits them. That's what JEP 506 means when it says scoped values "in the parent thread are automatically inherited by child threads created with StructuredTaskScope".[11]
- `fork()` asks the `ThreadFactory` for a new, unstarted thread and starts it *inside* the flock. If the owner's scoped-value bindings have changed since the scope opened, starting the thread throws a `StructureViolationException`.
- `join()` calls the flock's `awaitAll()`. Each finishing thread decrements a counter, and when it reaches zero the owner is woken with `LockSupport.unpark`. Then the scope asks the joiner for the result, or calls the joiner's `timeout()` method if the timer fired.
- **Cancellation** (a failed subtask, a joiner saying "done", or a timeout) shuts the flock down, interrupts every thread in it and wakes the owner. A subtask that finishes *after* the scope has been cancelled has its result dropped and stays `UNAVAILABLE`.

- **Timeouts** are a scheduled task on the common `ForkJoinPool` that cancels the scope when it fires.
- `close()` cancels the scope if you never joined, then closes the flock, which waits for every thread. If you close an outer flock while an inner one is still open, the inner ones are closed first (newest first) and a `StructureViolationException` is thrown.

What sits under a StructuredTaskScope



From StructuredTaskScopeImpl.java and jdk/internal/misc/ThreadFlock.java at openjdk/jdk commit cd37d2b.

The flock also registers a **thread container** *≈ the JDK's internal record of which threads belong to which group, used by tools such as jcmd's JSON thread dump*, which is why a JSON thread dump can show subtasks nested under the scope that owns them. ThreadFlock's own documentation says flocks are "conceptually nodes in a tree": a thread in flock A that opens flock B makes A the parent of B.[10]

That's the "lower-level" layer. At Devoxx Belgium this month, Alan Bateman and Viktor Klang gave a talk called "Structured Concurrency Looms". Its Inside.java description says they look at "a new lower-level API behind the latest design", and that this work is "expected to form the basis of a future lower-level structured concurrency API".[12] The talk description doesn't name ThreadFlock, so we can't say for sure that's what they mean. But it's a clear signal that `StructuredTaskScope` is the friendly front door, and more building blocks may follow.

WHAT IT MEANS FOR YOU

If you're on JDK 21 or 25 LTS

Nothing changes for you today. The final API will be part of every JDK from 28 onwards, and the next LTS that includes it is planned to be **JDK 29, in September 2027**, according to Oracle's support roadmap (which notes LTS designations "are subject to change").[13] Until then, `ExecutorService` and `CompletableFuture` remain the production tools on LTS releases.

If you experimented with the preview on an LTS release, be aware that the API you used is **not** the final one. The JDK 21 preview (constructors, `ShutdownOnFailure`) is entirely different, and the JDK 25 preview is close

but not compatible. We checked the JDK 25 source: there, `StructuredTaskScope` has two type parameters, `allSuccessfulOrThrow()` returns a `Stream` of subtasks, the "first success" joiner is called `anySuccessfulResultOrThrow()`, failures arrive as an unchecked `FailedException`, and `open(joiner, ...)` takes a `Function` rather than a `UnaryOperator`. JDK 25 also had an `awaitAll()` joiner, which is gone.[14,1]

Collect all results: JDK 25 preview vs final

JDK 25 preview (JEP 505)

```
try (var s = StructuredTaskScope.open(
    Joiner.<Price>allSuccessfulOrThrow())) {
  tasks.forEach(s::fork);
  return s.join()            // Stream
      .map(Subtask::get)
      .toList();
} // failure: FailedException
```

JDK 28 final (JEP 543)

```
try (var s = StructuredTaskScope.open(
    Joiner.<Price>allSuccessfulOrThrow())) {
  tasks.forEach(s::fork);
  return s.join();           // List<Price>
} // failure: ExecutionException
```

Signatures from StructuredTaskScope.java at tag jdk-25-ga and at openjdk/jdk commit cd37d2b. 'tasks' is a list of Callable<Price>. JDK 25 code needs --enable-preview.

If you're on the six-monthly train

If you run JDK 27 with `--enable-preview` for structured concurrency (and nothing else), moving to JDK 28 means **removing the flag** from your compiler settings, test runner, run scripts and container images. That's the whole migration. In issue #5 we described how the open-source media server streamarr was planning exactly that: its tracking issue lists the build, the tests, the Procfile, the container image and CI.[15]

If you maintain a library

This is the group the change really unlocks. Until now, a library that used `StructuredTaskScope` would only load on one JDK version, with a flag. From JDK 28, you can use it the way you use `CompletableFuture`. One small project already shows the pattern for bridging the gap: **velo/structured-concurrency** repackages the final JDK 28 API under its own package name as a plain library "for JDK 21 or later, without `--enable-preview`", built from the public Javadoc and JEP text rather than OpenJDK's code. It ships an **OpenRewrite** ≈ *an open-source tool that applies automated, rule-based refactorings ("recipes") to source code* recipe that renames the package to `java.util.concurrent` once you move to JDK 28. Its README lists known gaps, for example subtasks don't inherit scoped-value bindings.[16] It's a tiny project (no stars when we looked), so treat it as an illustration of the migration path, not an endorsement.

If you write custom joiners

Read the three changes above. In particular, `onComplete()` can now be called on the owner thread, during `fork()`, for a subtask that never ran. A joiner that assumed `onComplete()` always runs on a subtask's own thread, or that every completed subtask actually executed, should be checked.

💥 Watch out for

- **It's in the main line, not in a build yet.** JDK 28 early-access build 20 (15 October) predates the commit. Wait for the next build before testing.[4]
- **Preview code doesn't carry over automatically.** Classes compiled with `--enable-preview` on JDK 27 won't load on JDK 28. You must recompile, even though the source is the same.[7]
- **JDK 21 and JDK 25 preview code needs real changes**, not just a recompile (see the comparison above). Old tutorials using `ShutdownOnFailure`, `FailedException` or `awaitAll()` describe earlier previews.
- **Cancellation is still cooperative.** Cancelling a scope interrupts the subtask threads; a task that ignores interrupts can still hold `close()` open.[1]
- `Subtask.get()` **never blocks.** It throws unless the owner has joined and the subtask succeeded. Call it after `join()`.[1]

WHAT HAPPENS NEXT

The next JDK 28 early-access build should be the first one with the final API. JDK 28 enters **Rampdown Phase One** *≈ the point in a JDK release's schedule when the feature set is frozen and only bug fixes go in* later in its schedule and ships in March 2027.[5] Meanwhile, keep an eye on the Loom project for the "lower-level structured concurrency API" that Bateman and Klang previewed at Devoxx.[12] For most application developers, though, the decision point is JDK 29: the first planned LTS where you can use structured concurrency with no flags and no caveats.[13]

📖 Words from this story

JEP 543: the proposal that makes structured concurrency final.

Proposed to Target: a JEP formally proposed for a release, followed by a short review.

Main line: the openjdk/jdk master branch the next release is built from.

LTS: a long-term-support JDK release.

OpenRewrite: an automated refactoring tool driven by recipes.

Early-access build: a weekly pre-release JDK binary.

Thread: an independent path of execution.

Structured concurrency: tasks started in a block finish before the block ends.

Scope: a block that owns a group of concurrent tasks.

Subtask: work forked inside a scope, on its own thread.

Owner: the thread that opened a scope.

Virtual threads: cheap, JVM-managed threads.

Preview API: complete but not permanent; needs `--enable-preview`.

Minor version 65535: the class-file marker for code that uses preview features.

Joiner: the policy object that decides when a scope is done and what `join()` returns.

UnaryOperator: a function from a type to the same type.

ThreadFactory: creates the threads a scope runs subtasks in.

Suppressed exception: an extra exception attached to the main one.

ThreadFlock: the internal JDK class that groups and waits for a scope's threads.

Scoped value: an immutable value inherited by a scope's subtasks.

Thread container: the JDK's internal record of thread groupings, used by thread dumps.

Rampdown Phase One: the feature freeze in a JDK release schedule.

STORY 2

Jakarta EE gets a standard for instant startup

JAKARTA EE · ECLIPSE FOUNDATION · JAKARTA CRAC 1.0 · CREATION AND PLAN REVIEW APPROVED 13 OCT 2026 · ~14 MIN READ

Imagine your application server booting the way your laptop wakes from sleep. No scanning JARs, no wiring thousands of beans, no waiting for the JIT compiler to warm up: just pick up exactly where a previous run left off. That's the promise of checkpoint and restore, and several Java runtimes already offer it, each in its own way. Last week the Jakarta EE community voted to turn it into a standard. Here's what a JVM snapshot actually is, why it's harder than it sounds, and what the new Jakarta CRaC specification sets out to fix.

WHAT HAPPENED

On 13 October 2026, the Jakarta EE Specification Committee's ballot on a new specification, **Jakarta CRaC 1.0**, "concluded successfully". It was a combined **creation review** ≈ *the Eclipse Foundation vote that approves starting a new specification project* and **plan review** ≈ *the vote that approves what a specification's next release intends to deliver*, with +1 votes from the representatives of Fujitsu, IBM, Oracle, Payara, Tomitribe and others, and no votes against.[17] Eclipse Foundation's Ivar Grimstad merged the specification's page into the official Jakarta EE specifications repository on 14 October.[18]

The spec's description is a single sentence: Jakarta CRaC "defines how Jakarta EE runtimes take part in JVM checkpoint and restore, so that an application can be restored from a snapshot instead of starting from scratch." Its stated goal is "to cut startup time substantially while keeping applications correct and portable across compliant application servers".[19]

The project proposal was published on 17 September. Its project lead is Gerrit Grunwald of Azul, the company behind the original CRaC work,[30] and its initial committers are Ivar Grimstad, Steve Butler, Rustam Mehmandarov and Thomas Watson. Azul and IBM are listed as interested parties.[20] Grimstad's weekly "Hashtag Jakarta EE" column announced the community review on 27 September.[21]

This is a brand-new specification, so there's no code to download yet. It's a commitment, approved by the vendors who build Jakarta EE servers, to standardise something they each currently do differently.

THE BACKGROUND YOU NEED

Jakarta EE in two minutes

Jakarta EE ≈ *the open, vendor-neutral set of enterprise Java specifications (formerly Java EE) run by the Eclipse Foundation: CDI, Persistence (JPA), REST, Servlet and many more* isn't a product. It's a set of **specifications** ≈ *documents plus an API JAR plus a test suite that define what a feature must do, so several vendors can implement it compatibly*. Products such as GlassFish, Open Liberty, WildFly or Payara become "Jakarta EE compatible" by passing the **TCK** ≈ *Technology Compatibility Kit: the official test suite an implementation must pass to claim compatibility*. Even if you only use Spring, you use Jakarta EE specs every day: `jakarta.persistence`, `jakarta.servlet` and `jakarta.validation` are all Jakarta specifications.

The platform comes in three sizes. The **Core Profile** ≈ *the smallest Jakarta EE profile, aimed at microservices and smaller runtimes: CDI Lite, REST, JSON and annotations*, the **Web Profile** (adds Servlet, Persistence, Security, Faces and more) and the full **Platform**.[22]

Why Java starts slowly

When a Java application starts, the JVM loads and links thousands of classes, your framework scans for components and wires them together, and the code runs in the slow **interpreter** ≈ *the JVM's mode for running bytecode one instruction at a time, before it has compiled anything* until the **JIT compiler** ≈ *the just-in-time compiler that turns frequently used bytecode into optimised machine code while the program runs* has seen enough to optimise the hot paths. That last stage is called **warm-up** ≈ *the period after startup during which the JVM is still collecting profiles and compiling hot code, so performance is below its peak*. For a long-running server, a few seconds of startup hardly matters. For a container that scales from zero, or a serverless function, it's the whole problem.

There are three broad ways to attack it, and they're all live in the Java world today:

1. **Do the work ahead of time, at build time.** GraalVM's **native image** ≈ *an ahead-of-time compiler that turns a Java application into a standalone native executable that starts almost instantly, with some restrictions on dynamic features* compiles everything before you ship.
2. **Cache the work from a training run.** **Project Leyden** ≈ *the OpenJDK project improving startup and warm-up by shifting work into an AOT cache produced by a training run* records what a training run loaded, linked (and, from JDK 28, compiled) into a cache file that the next run reuses. We covered its JDK 28 step, JEP 544, in issues #2 and #10.
3. **Snapshot the running process itself.** That's **CRaC** ≈ *Coordinated Restore at Checkpoint: an OpenJDK project that saves a running JVM to disk and restores it later, after giving the application a chance to prepare*.

CRaC: save the whole JVM, then come back

CRaC began as an OpenJDK project sponsored by the HotSpot Group. Its goal is to "develop a new standard mechanism-agnostic API to notify Java programs about the checkpoint and restore events".[23] The idea is simple. You start your application, let it boot (and optionally warm up), then take a **checkpoint** ≈ *a snapshot of a running JVM process, including its heap and compiled code, written to disk*. Later you **restore** ≈ *start a new process from a checkpoint instead of running main() from scratch* from those files and continue as if nothing had happened, already booted and already warm. Introducing the project on foojay.io in 2022, Grunwald credited Azul's Anton Kozlov with the proposal and described the result as cutting startup "from hundreds of seconds to tenths of milliseconds".[24]

On Linux, the snapshot is typically taken with **CRIU** ≈ *Checkpoint/Restore In Userspace: a Linux tool that can freeze a running process and save its full state to files*. You opt in with JVM flags and trigger the snapshot with **jcmd** ≈ *the JDK's command-line tool for sending diagnostic commands to a running JVM*:[25]

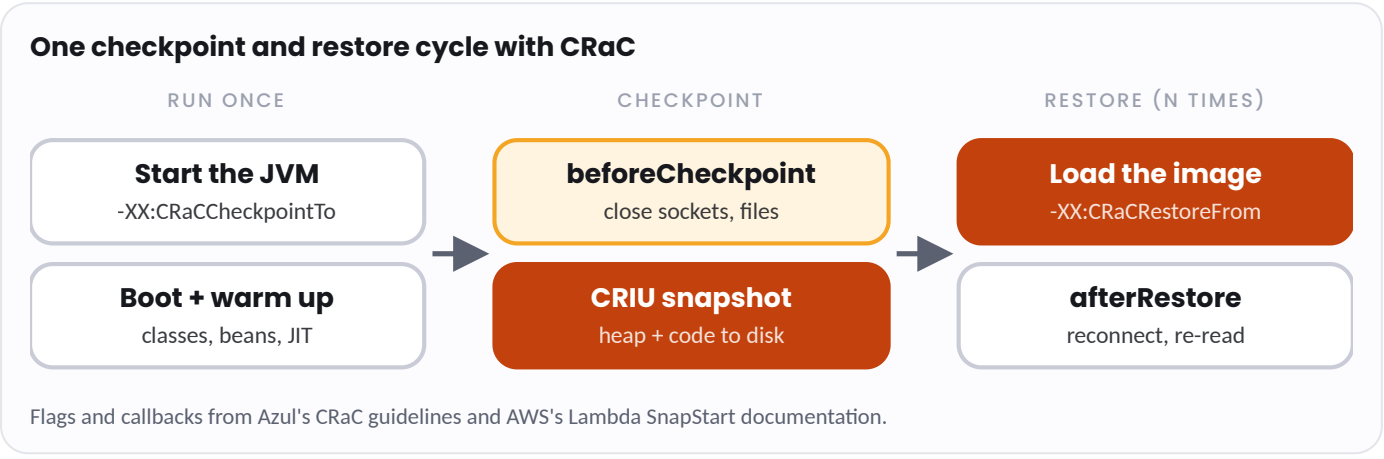
```
# 1. Run normally, but allow checkpoints to be written to ./cr
java -XX:CRaCCheckpointTo=./cr -jar app.jar
# 2. From another terminal, take the checkpoint (the process exits)
jcmd app.jar JDK.checkpoint
# 3. Later, and as many times as you like: restore
java -XX:CRaCRestoreFrom=./cr
```

The "coordinated" part is the clever bit, and the reason a specification is needed. A process image can't contain things that only make sense at the moment it was taken: an open database connection, a listening socket, a file handle, a cached hostname or a random seed. So CRaC gives applications callbacks. You implement the **Resource interface** ≈ *org.crac.Resource: two methods, beforeCheckpoint() and afterRestore(), that let code close and reopen external resources around a snapshot* and register it:[26]

```
// org.crac:crac library; works with a CRaC-enabled JDK
public class Pool implements Resource {
    public Pool() { Core.getGlobalContext().register(this); }

    @Override public void beforeCheckpoint(Context<? extends Resource> c) {
        closeAllConnections();   // nothing external in the image
    }
    @Override public void afterRestore(Context<? extends Resource> c) {
        reconnect();             // fresh connections after restore
    }
}
```

If something is still open when you ask for a checkpoint, the JVM refuses: the foojay introduction notes the implementation "throws exceptions if resources such as open sockets are still open at checkpoint time".[24] The `org.crac` library has a nice escape hatch, too: on a normal JDK it falls back to a dummy implementation, so your app runs normally and only checkpoint requests fail.[25]



Everybody already does it, differently

That's the problem Jakarta CRaC is trying to solve. Its proposal opens by noting that "many of the Jakarta EE implementations support checkpoint and restore today."[20] Some examples:

- **Open Liberty InstantOn** (IBM) checkpoints a Liberty server using the **OpenJ9** ≈ *Eclipse OpenJ9, an alternative JVM originally developed by IBM, used by IBM Semeru runtimes* JVM and CRIU. Its 2022 announcement reported startup savings "up to 90%", depending on the application, and offered checkpoint points such as `beforeAppStart` and `afterAppStart`.[27]
- **Spring Framework** integrates with Project CRaC: before a checkpoint it stops all running beans through `Lifecycle.stop`, restarts them after restore, and with `-Dspring.context.checkpoint=onRefresh` it can take a checkpoint automatically during startup.[28]
- **AWS Lambda SnapStart** for Java uses the same `org.crac` hooks so your function can prepare for its snapshot and clean up after restore.[26]
- **Helidon** publishes a CRaC extension: its separate extensions repository has a `crac/27.0.0` release tag.[29]

Each of these speaks to the JVM through the same low-level `Resource` callbacks, but each defines its own rules for **when** the server takes the snapshot and **how** the application's components are told about it. If you write a Jakarta EE application that holds a **JMS** ≈ *Jakarta Messaging (formerly Java Message Service), the standard API for message queues and topics* connection or a scheduled timer, you currently have to know which server you're on.

HOW IT WORKS

What Jakarta CRaC will specify

The specification page, approved last week, describes two ways of creating a checkpoint:[19]

- **Automatic checkpoint.** "The checkpoint is taken without user involvement, directly after the application server has been loaded and right before the application is loaded." Later starts restore from that snapshot, "so the server initialization cost is paid only once."
- **Manual checkpoint.** "The user triggers the checkpoint from outside the JVM, for example with `jcmd`." Here the application server "implements the CRaC Resource interface and acts as the single entry point for the JVM."

That second sentence is the key design decision. Instead of every library and every bean registering its own low-level CRaC `Resource`, the **server** registers once. When the JVM calls the server's `beforeCheckpoint()`, the server "informs all of its own critical resources, such as file handles and socket connections, so they can move into a safe state". The spec says this happens "through CDI events and annotations". After a restore, the JVM calls the server's `afterRestore()` and the server notifies the same resources "through the same mechanism".[19]

CDI ≈ *Jakarta Contexts and Dependency Injection: the Jakarta EE standard for beans, dependency injection, scopes and events* events are a natural fit. You already observe lifecycle events in CDI, such as the application scope being initialised or destroyed. Grimstad and Mehmandarov's 2025 Devnexus talk "Duke on CRaC with Jakarta EE" showed exactly that kind of bridge: an `@ApplicationScoped` CDI bean implementing `org.crac.Resource`,

alongside CDI's `@Initialized` and `@Destroyed` lifecycle events, with a slide floating Jakarta CRaC as "an idea".[30] Eighteen months later, the idea is an approved project.

Who talks to whom under Jakarta CRaC (as proposed)

JVM with CRaC support	calls beforeCheckpoint / afterRestore
Application server	the single registered CRaC Resource
CDI events and annotations	standard notifications, with ordering
Your beans and server resources	close sockets/files, then reopen

Based on the Jakarta CRaC 1.0 specification page (jakarta.ee specifications repository, Oct 2026). The exact annotation and event names aren't defined yet.

The plan lists six deliverables: define the automatic checkpoint ("when it occurs in the server lifecycle and what state it may contain"); define the manual flow; define the server's obligation to implement the CRaC `Resource` interface and delegate to its own resources; define "the CDI events and annotations used to notify resources before a checkpoint and after a restore, including ordering"; define what happens "when a checkpoint or restore fails or is unavailable"; and provide a TCK.[19]

Just as important is what it **won't** do. It "does not mandate a particular checkpoint/restore implementation", so CRIU-based CRaC, OpenJ9's approach and future mechanisms can all qualify.[19] And the proposal states that it "does not seek to replace the existing CRaC API", and aims to work with both CRaC and InstantOn.[20] Like the rest of the next generation of Jakarta specs, it requires **Java SE 21 or higher**.[19]

CRaC versus Leyden: rivals or partners?

If you followed our Leyden coverage, you might wonder why Jakarta EE needs this when the JDK itself is getting faster at starting. The two approaches make different trade-offs, and InfoQ's 2025 comparison lays them out clearly.[31]

📸 CRaC (checkpoint/restore)

- Snapshots the whole running process
- Restores already booted and warm
- App must close and reopen files and sockets
- Linux only; needs a CRaC-enabled JDK
- Image may hold secrets in clear text

VS

🗄️ Leyden AOT cache

- Caches classes (and, in JDK 28, compiled code)
- Every start still runs main() normally
- No application code changes
- Cross-platform, part of standard OpenJDK
- Smaller gains than a full snapshot

In InfoQ's numbers, the Java 24 AOT cache started Spring PetClinic "up to 40% faster", and Helidon measured 67% and 62% speed-ups for Helidon SE and MP. CRaC, by contrast, gave Helidon 95% and 98%. But CRaC needs applications to close "all files and network connections" before the snapshot, "only runs on Linux", and its snapshot "may pose security risks, as it contains passwords and credentials in clear text".[31] Spring's own documentation carries the same warning: checkpoint files "may contain secrets and other sensitive data".[28]

So they're partners as much as rivals. Leyden is the low-risk default that every app gets almost for free. CRaC is the high-reward option for workloads where startup time is the product, such as scale-to-zero services and serverless functions, and it's exactly the kind of option that benefits from a standard, because getting resource handling right is what makes it safe.

WHAT IT MEANS FOR YOU

Today: nothing to change, something to watch

Jakarta CRaC 1.0 has no API, no implementation and no release date yet. Its "compatible implementations" section simply says "TBD".[19] It also isn't on the list of possible new specifications for Jakarta EE 12, which names Jakarta Config, HTTP, MVC, NoSQL and Query.[22] Our reading is that it will start life as a standalone specification that servers can adopt, rather than a required part of the EE 12 platform, but the spec page doesn't say either way.

If you already use checkpoint/restore, the practical advice is the same as before: make your resource handling explicit. Close connections, files and sockets before a checkpoint and reopen them afterwards, avoid caching things that go stale (hostnames, credentials, timestamps) and never ship a checkpoint image somewhere you wouldn't ship your secrets. Spring adds one subtle gotcha: tasks scheduled with `@Scheduled(fixedRate = 5000)` will run every missed execution at once after a restore, so it recommends `fixedDelay` or cron expressions instead.[28]

> 🌍 **In the real world**
>
> The most common place Java developers meet CRaC today isn't an application server at all, it's **AWS Lambda SnapStart**. SnapStart snapshots your initialised function and restores it later, and your code uses the standard `org.crac` hooks (the docs use `org.crac:crac:1.4.0`), and AWS sets hard limits: restoring plus your `afterRestore()` hooks must finish within 10 seconds, or the invocation fails with a `SnapStartTimeoutException`. The docs also warn that the global context only keeps a weak reference to your `Resource`, so if you don't hold on to it yourself, it can be garbage-collected and its hooks will silently never run.[26] That's exactly the kind of sharp edge a server-level standard can hide from application code.

💥 **Watch out for**

- **Approved doesn't mean available.** Jakarta CRaC is a newly created project with no API, TCK or implementation yet.[19]
- **Not part of EE 12 (as listed).** It doesn't appear among the EE 12 candidate specifications. Don't plan an EE 12 upgrade around it.[22]
- **CRaC still needs a CRaC-capable JVM and Linux.** A standard at the Jakarta level doesn't change what the JVM underneath has to support.[25,31]
- **Snapshots contain memory.** Treat checkpoint images like secrets.[28]
- **CDI 5.0 renamed its Maven artifact.** If you depend on the CDI API directly, update the coordinates when you move to it.[35]

Where Jakarta EE 12 stands

The CRaC vote lands in the middle of a busy release. In late September, the Jakarta EE Platform call firmed up the Jakarta EE 12 dates: **Core Profile on 1 December**, **Web Profile on 31 March** and the full **Platform on 15 May**.[32] Grimstad's 11 October column confirmed that the Core Profile date "is still December 1", timed for the JakartaOne Livestream.[33] The whole release requires **Java SE 21 or higher**, and compatible implementations must pass on Java 21 and Java 25.[22,34]

Jakarta EE 12 and Jakarta CRaC



Sources: jakartaee/specifications repository (ballot dates for Platform 12, CDI 5.0 and Jakarta CRaC); Jakarta CRaC proposal page; InfoQ, 5 Oct 2026; Hashtag Jakarta EE #354.

A few EE 12 pieces are worth knowing even if you never touch an application server, because Spring and Hibernate users depend on them too:[22,35,36]

- **CDI 5.0 is already final** (release ballot passed 11 September). It adds `@Eager` for eagerly initialised application-scoped beans and `@AutoClose` to call `close()` on `AutoCloseable` beans when they're destroyed. Its Maven coordinates change from `jakarta.enterprise:jakarta.enterprise.cdi-api` to `jakarta.cdi:jakarta.cdi-api`, with relocation artifacts to ease migration.
- **Jakarta Persistence 4.0** (the next JPA) is a major revision, "currently under very active development". Its long list of changes includes a new `EntityAgent` for working with detached entities, separate `Statement`

types for queries that modify data, `findMultiple()` for batch loading by id, an option to make `@ManyToOne` and `@OneToOne` lazy by default, and new lifecycle callbacks such as `@PreUpsert`. The JPQL definition moves into a new spec, **Jakarta Query**.

- **SecurityManager is gone.** "All Jakarta components in EE 12 no longer have Java SecurityManager usage in their APIs."
- **Still being decided:** whether Jakarta Authorization joins the Web Profile (a vote was "ongoing" on 11 October), and whether Jakarta NoSQL makes the Web Profile, which Grimstad has publicly argued for.[33,32]

WHAT HAPPENS NEXT

The Jakarta CRaC team now has to turn a two-paragraph plan into an API: which CDI events, which annotations, what ordering guarantees and what happens on failure. Watch the jakartaee specifications repository and the Jakarta EE Platform calls. Meanwhile, the Jakarta EE 12 Core Profile is due on 1 December, with the Web Profile and full Platform following in spring 2027.[32]

📖 Words from this story

Creation review: the vote that approves starting a new specification project.

Plan review: the vote that approves what a spec's next release will deliver.

Jakarta EE: the vendor-neutral set of enterprise Java specifications.

Specification: a document, API and test suite that vendors implement compatibly.

TCK: the official test suite for compatibility.

Core Profile: the smallest Jakarta EE profile, for microservices.

Interpreter: the JVM's slow, instruction-by-instruction mode before compilation.

JIT compiler: compiles hot bytecode to optimised machine code at run time.

Warm-up: the period before the JVM reaches peak performance.

Native image: GraalVM's ahead-of-time compiled executable.

Project Leyden: OpenJDK's AOT cache approach to faster startup.

CRaC: Coordinated Restore at Checkpoint, snapshot and restore for the JVM.

Checkpoint: a snapshot of a running JVM written to disk.

Restore: starting a new process from a checkpoint.

CRIU: the Linux tool that freezes and saves a process.

jcmd: the JDK's diagnostic command tool.

OpenJ9: an alternative JVM, originally from IBM.

JMS: Jakarta Messaging, the standard messaging API.

Resource interface: `org.crac.Resource`, with `beforeCheckpoint()` and `afterRestore()`.

CDI: Jakarta's standard for beans, injection, scopes and events.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **Oracle's October Critical Patch Update ships tomorrow, 20 October:** it's the first quarterly update where Oracle JDK 21 builds are offered under the OTN licence instead of the free NFTC (our issue #1).[37,38]
- **A javac bug in record compact constructors is fixed in JDK 28 (16 October):** declaring `final int c;` and assigning it later inside a compact constructor was wrongly rejected; we reproduced the "cannot assign a value to final variable" error on JDK 21.[39]
- **JMX's `OpenType.ALLOWED_CLASSNAMES` is now deprecated for removal in JDK 28 (16 October):** the array, deprecated since Java 6, gets `forRemoval=true`.[40]
- **Valhalla spec work: value objects are never finalized (15 October):** `Object.finalize`'s Javadoc now says a value class can't override `finalize` and the garbage collector never calls it on a value object.[41]
- **Inside Java Newscast #72 (17 October) tells the story of a critical JDK 27 bug caught in the release-candidate phase:** a change that could corrupt strings was backed out of JDK 27 and redone in the main line, a good look at how OpenJDK handles late bugs.[42]

---

1. **openjdk/jdk commit cd37d2b "8391460: Implement JEP 543: Structured Concurrency"**, Alan Bateman, reviewed by Viktor Klang, 17 Oct 2026. We read the full diff with git, compared the public declarations in `StructuredTaskScope.java` with its parent commit (identical), and read `StructuredTaskScopeImpl.java` and the new tests in `StructuredTaskScopeTest.java`.
2. **Java News Roundup: Grails 8, Open Liberty, GlassFish, Infinispan, LangChain4j, Spock**, InfoQ, 12 Oct 2026 (JEP 543 Proposed to Target; review to conclude 13 Oct).
3. **Inside.java** front page and **Alan Bateman's author page**, checked 19 Oct 2026 (latest structured concurrency "targeted" post: JEP 533 for JDK 27, 11 May 2026; JEP 545 targeted to JDK 28 on 16 Oct).
4. openjdk/jdk tags checked with git on 19 Oct 2026: **jdk-28+20** is the latest and points to commit 84054db (15 Oct), which predates cd37d2b.
5. **Java 28 starts to take shape**, Paul Krill, InfoWorld, 24 Sep 2026 (JDK 28 "due in March 2027", non-LTS).
6. Our own test on OpenJDK 21.0.12.1 (Ubuntu build) in the newsletter sandbox, 19 Oct 2026: a class using `StructuredTaskScope.ShutdownOnFailure`, compiled with `javac --release 21 --enable-preview`; `javap -v` reports minor version 65535, major 65; running without `--enable-preview` gives the error shown; with the flag it prints the expected result.
7. **JDK-8200440: Add JVM support for preview features**, OpenJDK bug tracker.
8. **JDK28 の構造化並行処理 - JEP 543: Structured Concurrency**, A Memorandum (mammb), 8 Oct 2026 (JEP history from 428 to 543).
9. **JEP 533 Tightens Exception Handling in Java's Structured Concurrency for JDK 27**, A N M Bazlur Rahman, InfoQ, 13 May 2026.
10. `jdk.internal.misc.ThreadFlock`, openjdk/jdk at commit cd37d2b (class Javadoc and `start` / `close` documentation, read with git).
11. **JEP 506: Scoped Values**, OpenJDK (delivered in JDK 25).
12. **Structured Concurrency Looms**, Alan Bateman and Viktor Klang, Devoxx Belgium 2026, Inside.java, 12 Oct 2026.
13. **Oracle Java SE Support Roadmap**, Oracle (Java 28 non-LTS March 2027; Java 29 LTS September 2027; "designation and dates are subject to change").
14. `StructuredTaskScope.java` **at tag jdk-25-ga**, openjdk/jdk (read with git: two type parameters, `FailedException`, `anySuccessfulResultOrThrow`, `awaitAll`, `Function` configuration).
15. **streamarr-server issue #482: Move to JDK 28 and adopt the final structured concurrency API without preview features**, GitHub.
16. **velo/structured-concurrency** README, GitHub, viewed 19 Oct 2026.
17. **Jakarta CRaC 1.0 specification page, "Ballots"**, jakartaee/specifications (commit **d75dea8**, 14 Oct 2026: ballot "concluded successfully on 2026-10-13").
18. **Combined Creation and Plan review for Jakarta CRaC, PR #931**, Ivar Grimstad, merged 14 Oct 2026 (commit 4f7cd38).

19. **Jakarta CRaC 1.0 (Under Development)**, jakartaee/specifications, dated 5 Oct 2026 (description, deliverables, minimum Java SE 21, compatible implementations "TBD"; read with git).
20. **Jakarta CRaC project proposal**, Eclipse Foundation, published 17 Sep 2026, last modified 15 Oct 2026.
21. **Hashtag Jakarta EE #352**, Ivar Grimstad, 27 Sep 2026.
22. **Jakarta EE Platform 12 (Under development)**, plus the **Core Profile 12** and **Web Profile 12** pages, jakartaee/specifications (read with git on 19 Oct 2026; plan reviews concluded 14 and 19 May 2025).
23. **CRaC Project**, OpenJDK.
24. **Introducing the OpenJDK Coordinated Restore at Checkpoint Project**, Gerrit Grunwald, foojay.io, 18 Apr 2022.
25. **CRaC guidelines**, Azul documentation (flags, `jcmd ... JDK.checkpoint`, Linux x64/Arm64, `org.crac` dummy fallback).
26. **Lambda SnapStart runtime hooks for Java**, AWS Lambda Developer Guide.
27. **Liberty InstantOn startup for cloud native Java applications**, Open Liberty blog, 29 Sep 2022.
28. **JVM Checkpoint Restore**, Spring Framework reference documentation (7.1).
29. **helidon-io/helidon-extensions** tags, checked with git on 19 Oct 2026 (`crac/27.0.0`).
30. **Duke on CRaC with Jakarta EE**, Ivar Grimstad and Rustam Mehmandarov, Devnexus 2025, Speaker Deck, 6 Mar 2025.
31. **InfoQ news on Java 24 and Project Leyden's JEP 483**, InfoQ, March 2025 (AOT cache vs CRaC vs GraalVM Native Image, with Helidon measurements).
32. **Java News Roundup: JobRunr 9, OpenXava 8, Quarkus, LangChain4j, JNoSQL, Introducing Lathe**, InfoQ, 5 Oct 2026 (Jakarta EE 12 dates; Authorization vote; NoSQL).
33. **Hashtag Jakarta EE #354**, Ivar Grimstad, 11 Oct 2026.
34. **Jakarta EE 12 Release Plan (draft)**, Jakarta EE Platform project (Java SE 21 minimum; ratification needs passing implementations on Java 21 and Java 25).
35. **Jakarta Contexts and Dependency Injection 5.0**, jakartaee/specifications (release review ballot concluded 11 Sep 2026; compatible implementation Weld 7.0.0.CR1).
36. **Jakarta Persistence 4.0 (under development)**, jakartaee/specifications (read with git, 19 Oct 2026).
37. **Critical Security Patch Update for September 2026 Now Available**, Oracle E-Business Suite Technology blog (next CPU dates: 20 Oct 2026, 19 Jan, 20 Apr, 20 Jul 2027).
38. **JDK 21 approaches end of permissive license**, Oracle Java blog ("beginning with the October 2026 Critical Patch Update (CPU)" under the OTN license).
39. **openjdk/jdk commit 75809ed "8380732: Final variables cannot be assigned separately from declaration in record compact constructor"**, Vicente Romero, 16 Oct 2026 (read with git; reproduced on OpenJDK 21.0.12.1).
40. **openjdk/jdk commit d0287a5 "8394269: Deprecate OpenType.ALLOWED_CLASSNAMES for removal"**, Kevin Walls, 16 Oct 2026.
41. **openjdk/jdk commit decb49e "8388161: value-objects-jls: 12.6: address finalization of value objects"**, Dan Smith, 15 Oct 2026.
42. **Episode 72 "OpenJDK Bug Fixing, JavaOne, and JDK 28" [IJN]**, Inside.java, 17 Oct 2026.